

Relatório: T3 IA - Extrator Estruturado com LLM, RAG e Validação

Brunna Moura, Carlos Cruz, Rafael Queiroz, Victor Bitencourt

Definição do Problema

O grupo 4 desenvolveu uma aplicação para o projeto 2 da especificação: um extrator estruturado com LLM e validação. A aplicação recebe documentos em PDF ou texto bruto, extrai informações relevantes em campos definidos e valida a saída antes de devolvê-la ao usuário.

O trabalho também incorpora RAG para apoiar a extração. O sistema ingere uma base documental, segmenta o texto em chunks, gera embeddings, armazena os vetores em um banco persistente e recupera trechos relacionados ao documento analisado. Esses trechos ajudam o modelo a normalizar, classificar e checar informações, sem trocar a extração estruturada por uma resposta livre.

Base Documental

O grupo escolheu como base documental o relatório da CPMI dos atos de 8 de janeiro de 2023. O documento possui mais de mil páginas, linguagem institucional, registros de depoimentos, menções a órgãos públicos, pessoas, datas e fatos distribuídos ao longo de várias seções. Esse volume atende ao requisito da disciplina de trabalhar com uma base externa de conhecimento que justifique ingestão, vetorização e recuperação semântica.

O domínio escolhido favorece dois tipos de teste. No fluxo de consulta RAG, o sistema precisa localizar trechos específicos e citar páginas. No fluxo de extração estruturada, o sistema precisa identificar tipo de documento, título, evento principal, datas, atores, organizações, fatos, evidências, categorias, fontes e nível de confiança.

Pipeline de Ingestão

A rota POST /api/ingest executa a ingestão documental. O usuário envia um PDF e seleciona o embedder. A aplicação grava o arquivo em diretório temporário, extrai texto página por página com PyMuPDF, segmenta o conteúdo com RecursiveCharacterTextSplitter, gera embeddings para os chunks e salva o resultado no PostgreSQL com pgvector.

A configuração padrão usa chunks de 2000 caracteres e overlap de 200 caracteres. O grupo escolheu esse tamanho após inspeção manual do documento da CPMI. Trechos curtos demais separavam atores, fatos e justificativas que apareciam no mesmo contexto discursivo. O overlap reduz perdas nas fronteiras entre chunks e mantém parte da continuidade textual usada na recuperação.

Arquitetura da Solução

O projeto usa Python 3.12, FastAPI e uma organização em portas e adaptadores. A camada `src/core/` concentra modelos de domínio, prompts, interfaces e o serviço de orquestração. A camada `src/infra/` implementa os adaptadores de banco, embeddings e LLM. A camada `src/api/` expõe as rotas HTTP, schemas e a interface web.

O `RAGService` concentra as operações principais:

- `embed_and_store`: recebe chunks, calcula embeddings e grava os vetores no banco.
- `query`: vetoriza a pergunta, recupera trechos similares e pede uma resposta ao LLM.
- `extract`: recebe o documento, recupera contexto opcional e pede um JSON estruturado ao LLM.

O armazenamento vetorial usa PostgreSQL com pgvector, acessado por SQLAlchemy. A aplicação define dois models ORM em `src/infra/models.py`: `ChunkModel`, usado pela tabela `chunk` com embeddings OpenAI-compatíveis de 768 dimensões, e `ChunkSpacyModel`, usado pela tabela `chunk_spacy` com embeddings spaCy de 300 dimensões. Cada model registra o texto do chunk, a página de origem e o vetor associado.

As migrations do Alembic criam as tabelas correspondentes no banco. Essa separação evita misturar vetores de dimensionalidades diferentes e permite comparar o comportamento do sistema com embeddings OpenAI-compatíveis e com embeddings locais do spaCy.

Recuperação Vetorial

A rota `POST /api/query` implementa o fluxo de pergunta e resposta fundamentada. O sistema transforma a pergunta em vetor, consulta o pgvector por similaridade e inclui os trechos recuperados no prompt. O LLM responde em português e informa as páginas usadas como fonte.

A rota `POST /api/extract` também usa recuperação vetorial quando `top_k` é maior que zero. Nesse caso, o sistema usa o texto do documento recebido como base para buscar chunks similares já ingeridos. A extração principal vem do documento enviado pelo usuário, enquanto os chunks recuperados funcionam como apoio para normalização, classificação e checagem.

Extração Estruturada e Validação

O fluxo principal do projeto 2 está em `POST /api/extract`. A rota aceita PDF, texto bruto ou ambos. Quando recebe PDF, a aplicação extrai o texto com `PDFLoader`. Em seguida, o serviço limita o texto enviado ao modelo, recupera contexto externo quando solicitado e monta um prompt que exige JSON válido.

O objeto de saída usa o seguinte formato:

```
{
  "document_type": null,
  "title": null,
  "main_event": null,
  "dates": [],
  "actors": [],
  "organizations": [],
  "facts": [],
  "evidence": [],
  "categories": [],
  "sources": [
    {"claim": "string", "page": 0}
  ],
  "confidence": "baixa"
}
```

A aplicação valida a resposta em camadas. Primeiro, tenta interpretar a resposta com `json.loads`. Se o modelo devolver texto fora do JSON, o serviço procura o objeto dentro da resposta. Se o JSON estiver malformatado, o sistema faz uma segunda chamada ao LLM para reparo. Depois disso, o serviço normaliza aliases em português, converte listas, ajusta fontes e valida o resultado com Pydantic.

Os campos obrigatórios são `document_type`, `title`, `main_event` e `facts`. A aplicação registra campos ausentes em `missing_required_fields` e classifica a saída como `valid`, `partial` ou `invalid`. O serviço ainda aplica regras determinísticas para preencher campos óbvios, como usar a primeira linha do documento como título ou reconhecer “relatório” nas primeiras linhas.

Interface Web

A aplicação oferece uma interface web em `http://localhost:8000/`, implementada com HTML, CSS e JavaScript em `src/api/static/`. A interface organiza o fluxo de demonstração do projeto 2 em uma ação principal: **Ingerir e extrair PDF**.

Esse botão envia o mesmo arquivo para `POST /api/ingest` e, em seguida, para `POST /api/extract`. O usuário seleciona o PDF uma vez, escolhe o embedder e o valor de `top_k`, e a tela mostra o resultado estruturado com status de validação, confiança, campos extraídos, fontes e quantidade de chunks usados como contexto.

A tela também mantém a ação **Extrair sem ingestão**, usada para texto bruto ou para testar um documento sem atualizar a base vetorial. A área secundária de consulta RAG permite fazer perguntas sobre os chunks já armazenados.

Modelos e Componentes Utilizados

O projeto usa o SDK `openai` como cliente OpenAI-compatível. No ambiente Docker, o LLM padrão roda no Ollama, com `LLM_BASE_URL=http://ollama:11434/v1` e modelo `phi4-mini:latest`. Essa escolha permite executar o projeto localmente e ainda preserva a possibilidade de trocar o provedor por OpenAI, Gemini, Groq ou outro serviço compatível.

Para embeddings, a configuração padrão usa `nomic-embed-text:latest` no Ollama, com vetores de 768 dimensões. O projeto também possui um embedder com `pt_core_news_lg` do spaCy, com vetores de 300 dimensões, usado para comparação.

As principais bibliotecas são FastAPI, SQLAlchemy assíncrono, asyncpg, pgvector, PyMuPDF, langchain-text-splitters, OpenAI SDK, Pydantic, Alembic e pytest. O ambiente completo roda com Docker Compose, usando serviços separados para API, PostgreSQL e Ollama.

Protocolo Experimental

O grupo montou um conjunto com 30 casos de teste em `test_cases.json`. Os casos incluem perguntas fáceis, médias, ambíguas, perguntas cuja resposta não aparece na base e prompts que testam limites da aplicação. Os scripts `run_evaluation.py` e `run_evaluation_k15.py` executam os testes com valores diferentes de `top_k`.

A comparação principal avaliou o comportamento do RAG com `top_k=5` e `top_k=15`. O critério de pontuação atribuiu 1 para respostas consideradas corretas e 0 para respostas incorretas ou insuficientes.

Resultados

Desempenho com `top_k = 5`

Categoria	Acertos	Total
Casos fáceis	3	6
Casos médios	4	7
Casos ambíguos	3	6
Base de conhecimento insuficiente	5	6
Limites da aplicação	5	5
Total	20	30

Desempenho com `top_k = 15`

Categoria	Acertos	Total
Casos fáceis	3	6
Casos médios	6	7
Casos ambíguos	2	6
Base de conhecimento insuficiente	6	6
Limites da aplicação	5	5
Total	23	30

O aumento de `top_k` melhorou o resultado total, de 20 para 23 acertos. A melhora apareceu nos casos médios e nas perguntas com base insuficiente. Nos casos ambíguos, o desempenho caiu, indicando que mais contexto também pode introduzir trechos concorrentes e dificultar a resposta.

Testes e Reprodutibilidade

O repositório inclui testes com `pytest`. Os testes unitários cobrem chunking, extração, validação, repositório e rotas HTTP. O arquivo `tests/test_static_ui.py` verifica se a interface web é servida em `/` e se os arquivos estáticos referenciam as rotas usadas no fluxo principal.

O grupo executou verificações direcionadas durante o desenvolvimento:

```
uv run pytest tests/test_static_ui.py -q
uv run ruff check `
    src\api\main.py `
    src\settings.py `
    tests\conftest.py `
    tests\test_static_ui.py
python -m compileall src tests
```

O grupo também validou manualmente o ciclo principal com um PDF de teste: a ingestão gravou chunks no banco e a extração seguinte retornou `context_chunks_used: 5`.

Análise Crítica

O experimento mostra que a recuperação vetorial ajuda, mas o valor de `top_k` precisa combinar com o tipo de pergunta. Perguntas factuais se beneficiam de trechos adicionais quando a informação aparece dispersa. Perguntas ambíguas sofrem quando o sistema recupera passagens relacionadas, mas sem foco suficiente.

A qualidade dos embeddings também limita o resultado. O embedder local do `spaCy` usa vetores estáticos e captura menos contexto semântico do que modelos baseados em Transformers. O embedder OpenAI-compatível com `nomic-embed-text` melhora a integração com o fluxo local em Docker, mas ainda depende da qualidade do chunking e da formulação da consulta.

Na extração estruturada, a principal dificuldade está na disciplina do LLM em retornar JSON válido e completo. A validação com Pydantic, o reparo automático e os fallbacks reduzem falhas operacionais. Mesmo assim, campos como **facts**, **evidence** e **sources** exigem revisão em documentos longos, porque o modelo pode selecionar fatos gerais e deixar de fora evidências mais específicas.

A interface web melhorou a demonstração do projeto. O fluxo com um único upload evita que o usuário ingira um PDF e depois precise enviá-lo de novo em outra área da tela. Essa decisão tornou a apresentação mais coerente com o projeto 2, pois a extração estruturada aparece como tarefa principal e a ingestão entra como etapa necessária para fornecer contexto RAG.

Considerações sobre Uso de IA Generativa

A equipe usou IA generativa como apoio em etapas pontuais. Agentes de IA ajudaram na criação de casos de teste, no apoio aos scripts de comparação de **top_k**, na geração do diagrama de arquitetura em DOT e na revisão de partes da documentação. O grupo revisou o código gerado, executou testes e ajustou o comportamento da aplicação.

Na etapa da interface, a discussão começou com a seguinte pergunta feita ao assistente:

baseado no contexto do AGENTS.md vc acha valido criar uma interface para esse projeto?

A partir dessa discussão, o grupo decidiu criar uma interface simples para a demonstração. Depois dos testes manuais, o grupo ajustou a tela para chamar `/api/extract` e consolidou o fluxo em **Ingerir e extrair PDF**, que executa `/api/ingest` antes de `/api/extract` com o mesmo arquivo.

Conclusão

O projeto entrega uma aplicação com LLM, base documental externa, ingestão, embeddings, armazenamento vetorial persistente, recuperação por similaridade, extração estruturada e validação. A arquitetura em portas e adaptadores facilita a troca de embedders e provedores de LLM, enquanto a interface web concentra o fluxo de demonstração em uma experiência mais direta.

Os experimentos indicam que o aumento de **top_k** melhora parte das respostas, mas também pode prejudicar perguntas ambíguas. Uma etapa futura do projeto deve avaliar a extração estruturada campo a campo, com métricas de completude, precisão das fontes e comparação entre embedders.

Referências

- Especificação do trabalho: docs/T3_Especificacao_LLMs_RAG_Validacao.pdf
- Documentação do pgvector: <https://github.com/pgvector/pgvector>

- Modelos spaCy em português: <https://spacy.io/models/pt>
- Ollama: <https://ollama.com/>
- FastAPI: <https://fastapi.tiangolo.com/>